

Objectifs du cours

- Notion de classe et d'encapsulation
 - Classes, méthodes, constructeur
 - héritage, structures de données
 - STL : Standard Template Library
- Conception orientée objets
 - Méthode générale de conception
 - Technique de conception des classes
- Entrées sorties et fichiers

Aujourd'hui

- Notion de classe et de méthode
 - Les classes
 - Les méthodes
- Constructeurs

Rappels sur les structures

- Types contenant plusieurs variables de type différent

```
struct Fraction
{
    int numerateur;
    int denominateur;
};
```

- Utilisation

```
Fraction f;
f.numerateur=1;
f.denominateur=2;
```

Exemple de programme

```
// Fichier fraction.h

struct Fraction
{
    int numerateur;
    int denominateur;
};

void saisirFraction(Fraction &f);
void afficherFraction(Fraction f);
Fraction ajouterFractions(Fraction f1, Fraction f2);
```

Exemple de programme

```
// Fichier fraction.cpp
#include<iostream>
#include "fraction.h"
using namespace std ;

void saisirFraction(Fraction &f)
{
    cin>>f.numerateur>>f.denominateur;
}
void afficherFraction(Fraction f)
{
    cout<<f.numerateur<<" / "<<f.denominateur<<endl;
}

Fraction ajouterFractions(Fraction f1,Fraction f2)
{
    fraction resultat;
    resultat.num=f1.numerateur*f2.denominateur
                +f2.numerateur*f1.denominateur;
    resultat.denominateur=f1.denominateur*f2.denominateur;
    return resultat;
}
```

Exemple de programme

```
// Fichier main.cpp

#include <iostream>
#include "fraction.h"

using namespace std;

int main()
{
    Fraction f1,f2;

    cout<<"Entrer une fraction f1 : ";
    saisirFraction(f1);

    f2.num=1;
    f2.den=2;

    f1=ajouterFractions(f1,f2);

    cout<<"Résultat : ";
    afficherFraction(f1);
    return 0;
}
```

Critique du programme précédent

- On doit se référer aux membres de la structure pour chaque opération
- Les fonctions qui manipulent la structure sont définies séparément de cette structure
- Elles ont toujours 1 paramètre qui est du type de la structure

Notion de classe

- On regroupe dans la même déclaration
 - Les données membres appelés aussi champs
 - Des fonctions associées à la classe appelées fonctions membres ou méthodes
- Lors de la définition des fonctions membres
 - Il y a un paramètre implicite portant le nom de la classe
 - Ce paramètre est la variable située devant l'opérateur "." lors de l'appel de la méthode

Déclaration de la classe

```
// Fichier fraction.h

class Fraction
{
    int numérateur_;
    int dénominateur_;

public:

    void saisir();
    void afficher();
    Fraction ajouter(Fraction f);

};
```

Remarques

- On a supprimé le mot "Fraction" devant le nom des fonctions
 - Il n'est plus utile puisque la fonction appartient à la classe Fraction
- On a supprimé un paramètre de type Fraction dans chaque fonction membre
 - Ce paramètre existe par défaut car la fonction est membre de la classe

Remarques

- Le mot clef "public" indique que les membres déclarés après seront accessibles de l'extérieur de la classe
- Par défaut, les autres membres sont privé c'est à dire qu'ils ne sont pas utilisables directement

Conventions de nommage

- Les noms de classes commenceront par une majuscule
- Les noms de fonctions, de variables et de champs commenceront par une minuscule
- S'il y a plusieurs mots, chaque nouveau mot commencera par une majuscule
- Les noms de champs se termineront par un caractère souligné pour les différentier des variables locales

Définition des méthodes

```
// Fichier fraction.cpp

#include<iostream>
#include "fraction.h"
using namespace std ;

void Fraction::saisir()
{
    cin>>numérateur_>>denominateur_;
}

void Fraction::afficher()
{
    cout<<numérateur_<<" / "<<
        <<denominateur_<<endl;
}
```

Définition des méthodes

```
Fraction Fraction::ajouter(Fraction f)
{
    Fraction resultat;
    resultat.numerateur_ = numerateur_ * f.denominateur_
                             + denominateur_ * f.numerateur_;
    resultat.denominateur_ =
        denominateur_ * f.denominateur_;
    return resultat;
}
```

Remarques

- Le nom de la méthode est précédé de `Fraction::`
 - Il faut rappeler que cette méthode est membre de la classe `Fraction`
 - C'est le nom complet de la méthode
- Le paramètre implicite n'a pas de nom dans les méthodes
 - C'est normal car ce nom ne figure pas dans la liste des paramètres de la fonction
 - Il n'est connu que lors de l'appel comme dans :
`f1.saisir( )`

Utilisation de la classe

```
// Fichier main.cpp

#include <iostream>
#include "fraction.h"

using namespace std;

int main()
{
    Fraction f1,f2;

    cout<<"Entrer une fraction f1 : ";
    f1.saisir();

    f2.numerateur_=1;    // Erreur membre privé
    f2.denominateur_=2;  // Erreur membre privé

    f1=f1.ajouter(f2);
    cout<<"Résultat : ";
    f1.afficher();
    return 0;
}
```


Vocabulaire

- Les variables f1 et f2 sont appelées des "objets"
 - Un objet est plus qu'une variable car il possède des méthodes
 - Il est donc capable de réaliser des actions
- L'appel d'une méthode est appelée "envoi de message"
 - f1.saisir() = envoi du message "saisir" à l'objet f1
 - Lors de cet appel, l'objet "f1" est transmis comme paramètre implicite à la méthode

Principe d'encapsulation

- On ne peut pas utiliser directement les membres privés d'une classe
- Cet accès est réservé aux méthodes
- Pourquoi ?
 - Faciliter l'évolution du code : on peut changer les champs et les méthodes sans avoir de répercussion sur le code
 - Faciliter l'utilisation de la classe : l'utilisateur n'a pas besoin de savoir ce qu'il y a à l'intérieur de l'objet
 - Eviter les effets de bords (modifications accidentelles)

Comment corriger le code ?

- Mauvaise solution : rendre les 2 champs de la classe publics :

```
// Fichier fraction.h

class Fraction
{
public:
    int numerateur_;
    int denominateur_;

    void saisir();
    void afficher();
    Fraction ajouter(Fraction f);
};
```

- Plus d'encapsulation !

Bonne solution

- Créer une méthode spécifique pour remplacer les instructions

```
f2.numerateur_=1;  
f2.denominateur_=2;
```

- Méthode d'initialisation :

```
// fraction.h  
class Fraction  
{  
...  
void initialiser(int n,int d);  
};
```

```
// main.cpp  
...  
f2.initialiser(1,2);  
...
```

```
// fraction.cpp  
void Fraction::initialiser(int n,  
    int d)  
{  
    numerateur_=n;  
    denominateur_=d;  
}
```

Constructeur

- En C++, il existe une méthode standardisée pour l'initialisation : le constructeur
- Un constructeur est :
 - Une méthode portant le nom de la classe
 - Qui peut avoir des paramètres
 - Qui n'a pas de type de retour
 - Qui est appelée automatiquement à la déclaration d'un objet (s'il existe)
 - Il peut en exister plusieurs mais ayant des paramètres différents

Nouvelle déclaration et définition de la classe

```
// Fichier fraction.h

class Fraction
{
    int numérateur_;
    int dénominateur_;

public:
    Fraction(int n,int d);
    void saisir();
    void afficher();
    Fraction ajouter(Fraction f);
};
```

```
// Fichier fraction.cpp
...
Fraction::Fraction(int n,int d)
{
    numérateur_=n;
    dénominateur_=d;
}
```

Utilisation du constructeur

```
#include "fraction.h"
#include <iostream>

using namespace std;

int main()
{
    Fraction f(1,2);

    cout<<"votre fraction : ";
    f.Afficher();

    return 0;
}
```

Paramètres transmis
au constructeur

```
Fraction f(1,2);  <=>  Fraction f;
                      f.Fraction(1,2);
```

Attention, on ne peut plus déclarer une fraction sans paramètres !!

```
Fraction f; // Erreur de compilation
```

- Si le constructeur existe, il doit être appelé
- Pour l'appeler il faut 2 paramètres !

Comment restaurer l'appel initial ?

- Deux solutions :
 - Prévoir des valeurs d'initialisation par défaut
 - Prévoir un autre constructeur sans paramètres

Les 2 solutions

```
// Fichier fraction.h

class Fraction
{
    ...

public:
    Fraction(int n=0,int d=1);
    ...
};
```

```
// Fichier fraction.h

class Fraction
{
    ...
public:
    Fraction();
    Fraction(int n,int d);
    ...
};
```

```
// Fichier fraction.cpp
...
Fraction::Fraction(int n,int d)
{
    numerateur_=n;
    denominateur_=d;
}
```

```
// Fichier fraction.cpp
...
Fraction::Fraction()
{
    numerateur_=0;
    denominateur_=1;
}

Fraction::Fraction(int n,int d)
{...}
```

Utilisation dans le programme

```
// Fichier main.cpp

#include <iostream>
#include "fraction.h"

using namespace std;

int main()
{
    Fraction f1;
    cout<<"Entrer une fraction f1 : ";
    f1.saisir();

    Fraction f2(1,2);

    f1=f1.ajouter(f2);

    cout<<"Résultat : ";
    f1.afficher();
    return 0;
}
```

Autres outils utiles en programmation objet

- Méthodes en ligne
- Passage d'objets en paramètre
- Sur-définition de méthodes
- Redéfinition d'opérateurs
- Méthodes "const"

Méthodes en ligne

- Méthodes définies lors de la déclaration de la classe (dans le .h)
- Analogues à des macro-commandes
 - Le code correspondant est recopié à chaque utilisation
 - Exécution très rapide
 - Augmentation de la taille du code pour les grosses méthodes
 - A utiliser pour les méthodes très simples réalisant un accès à un champ ou une affectation

Exemples de méthodes en ligne

```
// Fichier fraction.h

class Fraction
{
    int numerateur_;
    int denominateur_;

public:

    Fraction(int n=0,int d=1);
    int getNumerateur() {return numerateur_;}
    int getDenominateur() {return denominateur_;}
    void setNumerateur(int n) {numerateur_=n;}
    void setDenominateur(int d) {denominateur_=d;}
    void saisir();
    void afficher();
    Fraction ajouter(Fraction f);
};
```

Conventions de nommage

- Les méthode permettant d'accéder à la valeur d'un champ commenceront par « get »
- Les méthodes permettant de changer la valeur d'un champ commenceront par « set »
- D'une manière générale, une méthode ou une fonction commencera généralement par un verbe à l'infinitif
- Il est recommandé d'éviter les abréviations dans les noms de fonctions ou de variables

Exercice 1

Déclarer et définir une classe Vecteur pour représenter un vecteur de réels à 2 dimensions. Cette classe devra comporter :

- un constructeur avec des valeurs d'initialisation par défaut,
- 2 méthodes en ligne pour lire les composantes x et y
- une méthode pour afficher les composantes à l'écran
- une méthode calculant le produit scalaire entre 2 vecteurs.

a) Déclarer et définir la classe

b) Ecrire un programme illustrant l'utilisation des méthodes